EIE4121 Machine Learning in Cyber-security
Project Report

Spam Email Detection

Ku Wing Fung 18075712d Yu Hin Chung Nikko 17049674d

Introduction

Spam emails are unwanted junk emails sent out in bulk to an indiscriminate recipient list. Typically, spam is sent for commercial purposes, but spam emails can be dangerous because they could include malicious links such as viruses, worms, and phishing attacks. With the popularization of the internet, email spam has become a severe issue in cybersecurity.

To cope with this issue, Internet Service Providers and Email Inbox Providers have installed anti-spam software to filter spam emails. Traditional spam filters use word filters or whitelisting and blacklisting, which introduce delays since they require human efforts. They are also less effective than anti-spam software because the cost to create a spam email account is meager.

To further improve the solution, experts in the cybersecurity field have used different machine learning-based approaches to counter these unwelcome messages, including support vector machines, decision trees, and deep learning. In this report, we will discuss the proposed machine learning-based solutions from two papers.

The two papers we selected are “Email Spam Classification Using Gated Recurrent Unit and Long Short-Term Memory” (Paper 1) and “Effective Email Spam Detection System using Extreme Gradient Boosting” (Paper 2).

Overview

Both Papers proposed supervised learning to train a model in order to solve the problem.

Paper 1 proposed using Gated Recurrent Unit (GRU) and Long Short-Term Memory (LSTM) to achieve email spam classification. They are a type of recurrent neural network(RNN) layer often used to process sequential data such as text. LSTM layer uses special units in addition to standard units of RNN. GRU layer is a simplified version of the LSTM layer and cheaper to run, enhancing computational expensiveness. Both GRU and LSTM tackle the vanishing-gradient problem by allowing past information to be reinjected later, thus capable of learning order dependence in sequence prediction problems.

Paper 2 proposed using boosting to classify spam emails. Boosting helps the input dataset to reduce the errors made by previous training algorithms. First, the dataset is fed into a learning network. The trained dataset may have some errors. This dataset can be fed into another training model to reduce the misidentified models, and iterations can make the dataset very robust. During the training, the misidentified labels’ weights are incremented. Gradient boosting, however, optimizes the loss function of the weak learner (dataset + loss function in the model) by adding another new model to the learner, minimizing the loss. Extreme gradient boosting is an optimization and utilization of the Gradient Boosting algorithm. Previously, gradient boosting follows a sequential order: fed a dataset into the first model and then another model. Extreme gradient boosting introduces parallelization and cache optimization so the data training can be done faster and uses fewer resources.

Detailed discussion in the proposed Solution of Paper 1

Dataset and Preprocessing

The training dataset used is an existing dataset from Enron's energy company, which includes the topic and content of emails sent between 1999 and 2005. The dataset has 34519 records. Ham data is labeled as 0, while spam data is labeled as 1. Duplicates elimination and data balancing are applied, so the portion of the processed training dataset is 1:1.

Model Building

In this paper, a deep learning approach is used, and two different model architectures are proposed. The only difference is that one uses the LSTM layer while another one uses the GRU layer. The other parts remain the same. First, Word embedding is used to convert the text data into numerical vectors. The text data are tokenized, then vectorized using the glove840B word embedding with word length 70. Then pass to either 100 units of LSTM or 100 units of GRU. After that, two 1024 neuron dense layers with ReLU are used. Finally, to perform classification, a two neuron dense layer with softmax is used. Spatial dropout, recurrent dropout, and dropout are applied into corresponding layers to overcome overfitting, hence improving the model's performance. The total number of parameters is 57,476,974 for the LSTM model, 57,436,874 for the GRU model [1].

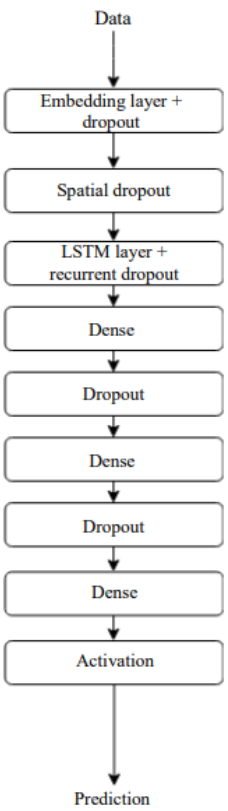


Fig. 4: LSTM model

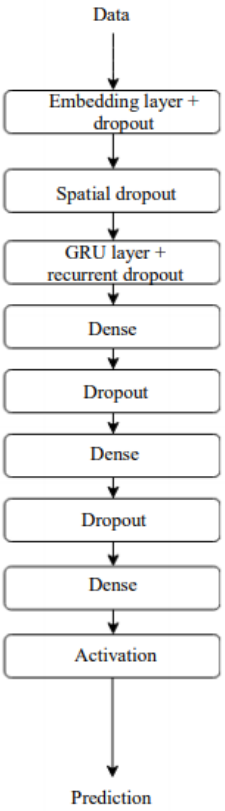


Fig. 5: GRU model

LSTM and GRU

Long Short Term Memory (LSTM) and Gated Recurrent Unit (GRU) are both variants of the Recurrent neural network (RNN) layer. RNN is a generalization of a feedforward neural network with an internal memory that can process sequential data such as letters and words. RNN layers take vectors as input, which we can get from the output of the word embedding layer. In RNN, we forward propagate, But we keep the information from the hidden layer for the second sample. The hidden layer for the second sample is a function of the new inputs, the updated weights, and the hidden layer from the previous iteration. RNN uses a tanh function as activation to regulate the output to be between -1 and 1.

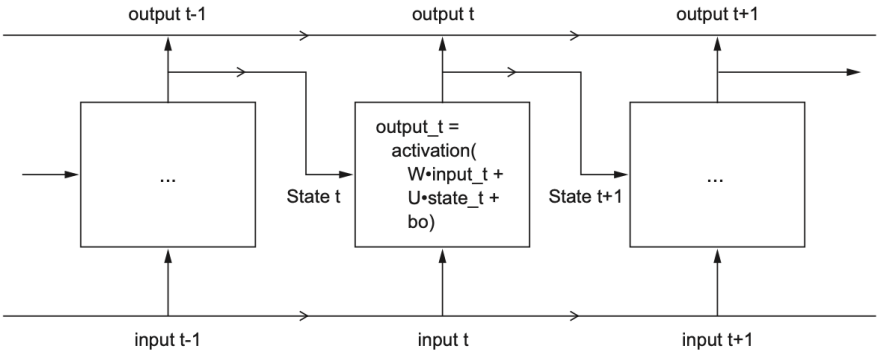
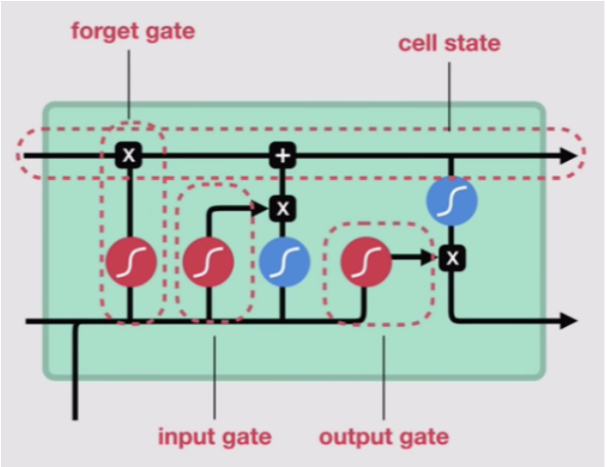
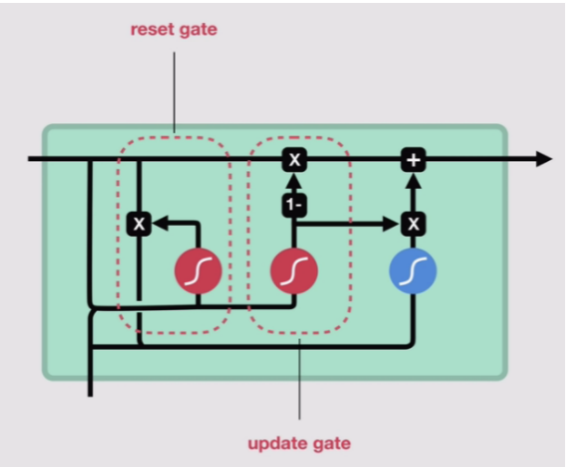


Figure 6.10 A simple RNN, unrolled over time

However, RNN will introduce the vanishing gradient problem because a significant change in the input will produce a slight change in the output due to the activation function [2]. Thus people have developed LSTM and GRU. LSTM has the same control flow as RNN, but the internal architecture is different. LSTM allows past information to be reinjected later, thus countering the vanishing gradient problem caused by RNN. A set of operations are used in the LSTM to allow it to have the ability to forget or keep the information. The gates contain sigmoid activations, which squishes values to be between 0 and 1. If a value passes through a sigmoid and becomes 0, it can be forgotten while keeping the others. Thus LSTM can choose the information to forget or remember during processing. GRU is a relatively cheap version of LSTM to enhance computational expensiveness. GRU has fewer tensor operations, but it still functions similarly to LSTM.



Long Short Term Memory (LSTM)



Gated Recurrent Units (GRU)

^The blue activation icons are tanh, while the red activation icons are sigmoid.

Model Training and Performance Measurement
The categorical cross-entropy is selected as the loss function because softmax activation is used as the last layer. Adam is used as the optimization algorithm. 7-Fold Cross-validation is used during training. Text data are split into seven groups. In the seven training scenarios, each scenario has a 6:1 ratio of training data to validation data. The split data was sent to train and test in each fold. For performance measurement, since the model's sensitivity is essential, accuracy and recall are used as the metrics.

Detailed discussion in the proposed Solution of Paper 2

Training Dataset and Preprocessing
Paper 2 used a benchmark spam dataset from the UCI machine learning repository, called the Spambase dataset [3]. It is a tabular dataset and already well feature engineered, and the dataset has 4601 records. The features are 57 continuous-valued defining features. Ham data is labeled as 0, while spam data is labeled as 1.

XGBoost
Paper 2 proposed a decision-tree-based ensemble model, using the software XGBoost XGBoost is based on tree boosting, and a simple review of tree boosting is described. The structure is like decision trees, but it contains continuous leaves (leaf) at each layer. The scores are added up for each entity in the function f. The final prediction produces a constant value for each entity [4].

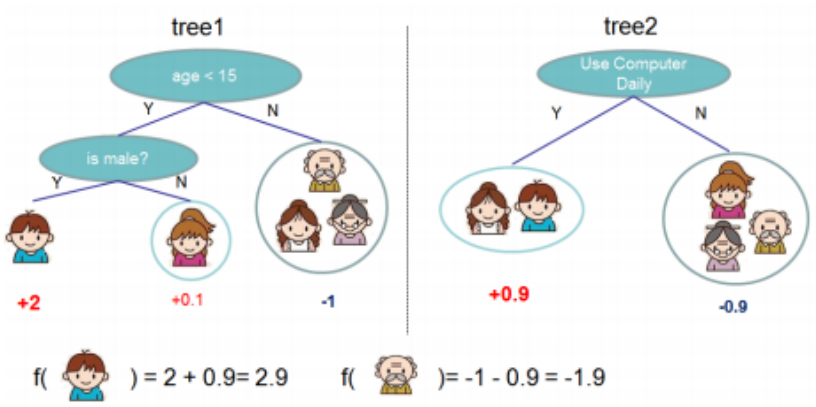


Figure 1: Tree Ensemble Model. The final prediction for a given example is the sum of predictions from each tree.

To produce the best decision trees programmatically, one needs to figure out the score of each possible structure of a tree. The smaller the score, the better the structure. The score can be calculated by pushing the g and h to the leaves and using the formula below to determine how good the decision tree is [5].

$$\text{obj}^* = -\frac{1}{2}\sum_{j=1}^{T}\frac{G_j^2}{H_j+\lambda} + \gamma T$$

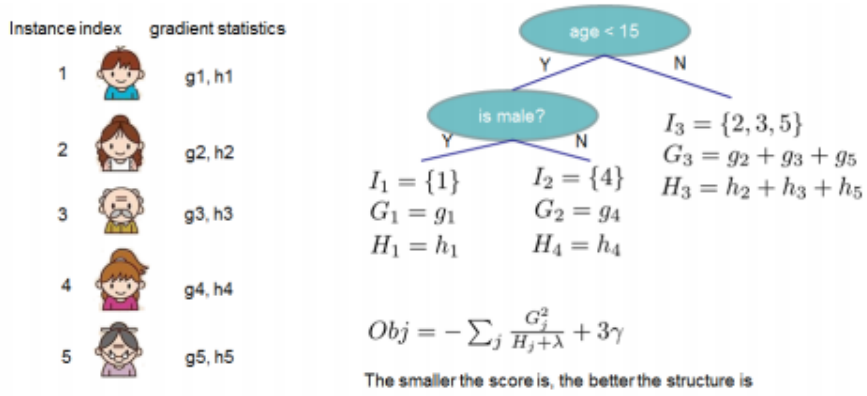
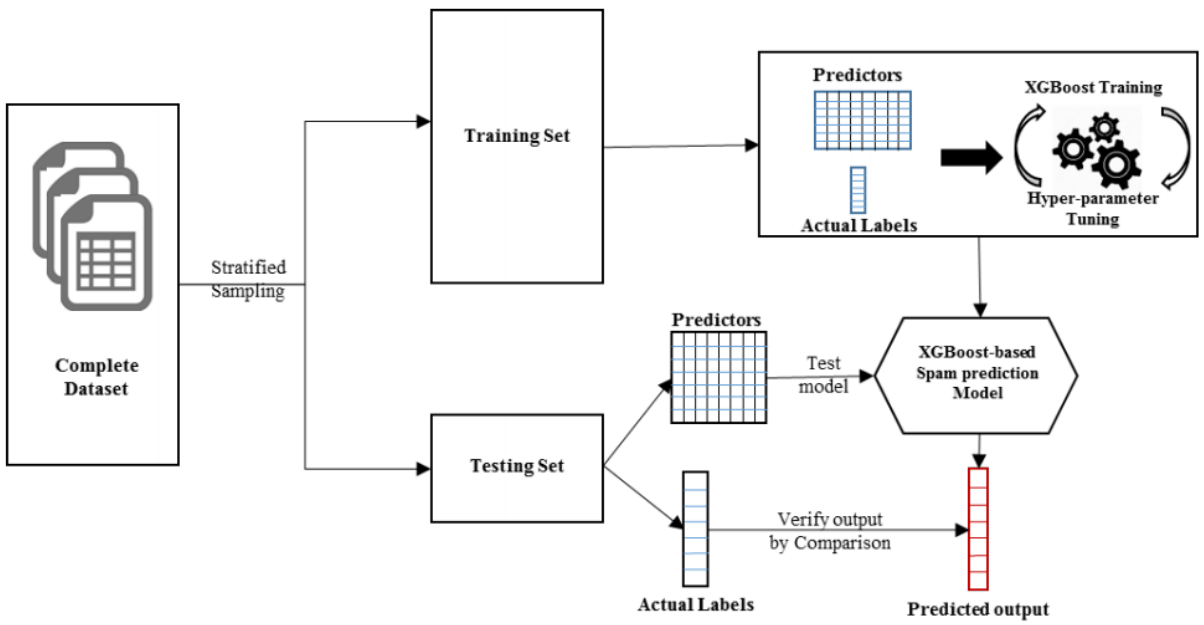


Figure 2: Structure Score Calculation. We only need to sum up the gradient and second order gradient statistics on each leaf, then apply the scoring formula to get the quality score.

By ad hoc method, we would like to enumerate all possible tree combines and compare its score, but this is not efficient. Instead, XGBoost uses the following formula to construct one optimized tree by predicting if each branch should be added to the XGBoost tree. If the value is smaller than γ, then do not add the branch [5].

$$Gain = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma$$

Model Training and Performance Measurement



The dataset is first split into a training set and testing set at a ratio of 7:3. XGBoost is used in the modeling process. Leave-one-out Cross-validation is used during training. The number

of folds equals the number of instances in the training set. This training approach takes a subset of the training set and then validates the rest in the dataset.
There are some procedures to find the hyperparameters, and they can be summarized as [4]:

1. Set hyperparameter values
2. Sequentially set a combination of hyperparameters and change one hyperparameter one at a time
    a. Train using the next set of hyper-parameters in the sequence step 2 and record performance
    b. Return step 2 if the list of hyperparameters are exhausted
3. Find the best combination of hyperparameters and use them to train the model

The optimal combination of hyperparameters found are Eta = 0.4, gamma = 0.2, maximum depth = 24, column sample = 0.75, number of rounds (early stopping after 10) = 200.

For the performance measurement, accuracy, specificity, recall, precision, F1 score, and balanced accuracy are used as evaluation metrics.

The Comparison

Dataset Features
In both datasets, they classify spam as 1 and non-spam as 0. Although they used the same way to label, the features data are entirely different.

Dataset of paper 1 is a text dataset. It is a lengthy email text sample and an attributed class to classify as spam (1) or non-spam (0). The feature is entirely raw email text data. No feature engineering is done to the dataset [1].

Table 1: Data sample

Email	Class
Subject: dobmeos with hgh my energy level has gone up ! stukm Introducing doctor - formulated hgh human growth hormone - also called hgh is referred to in medical science as the master hormone. it is very plentiful when we are young, but near the age of twenty - one our bodies begin to produce less of it. by the forty time we are nearly everyone is deficient in hgh and at eighty our production diminished has normally at least 90-95%. advantages of hgh : …	1

Dataset of paper 2 is a numerical dataset. There are a total of 57 continuous well-engineered features. The last column is the label, whether classified as spam (1) or non-spam (0). The features used are mostly lexical features. They are related to the percentage of words in the email that match a specific word, the percentage of characters in the email that match a specific unique character, and the total number of capital letters in the

email, and more. Detailed documentation can be found in the file 'spambase.DOCUMENTATION' at the UCI Machine Learning Repository [2].

[3]:
```
df = pd.read_csv('../input/spambase/realspambase.data', header=None)
df
```

[3]:

	0	1	2	3	4	5	6	7	8	9	...	48	49	50	51	52	53	54	55	56	57
0	0.00	0.64	0.64	0.0	0.32	0.00	0.00	0.00	0.00	0.00	...	0.000	0.000	0.0	0.778	0.000	0.000	3.756	61	278	1
1	0.21	0.28	0.50	0.0	0.14	0.28	0.21	0.07	0.00	0.94	...	0.000	0.132	0.0	0.372	0.180	0.048	5.114	101	1028	1
2	0.06	0.00	0.71	0.0	1.23	0.19	0.19	0.12	0.64	0.25	...	0.010	0.143	0.0	0.276	0.184	0.010	9.821	485	2259	1
3	0.00	0.00	0.00	0.0	0.63	0.00	0.31	0.63	0.31	0.63	...	0.000	0.137	0.0	0.137	0.000	0.000	3.537	40	191	1
4	0.00	0.00	0.00	0.0	0.63	0.00	0.31	0.63	0.31	0.63	...	0.000	0.135	0.0	0.135	0.000	0.000	3.537	40	191	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
4596	0.31	0.00	0.62	0.0	0.00	0.31	0.00	0.00	0.00	0.00	...	0.000	0.232	0.0	0.000	0.000	0.000	1.142	3	88	0
4597	0.00	0.00	0.00	0.0	0.00	0.00	0.00	0.00	0.00	0.00	...	0.000	0.000	0.0	0.353	0.000	0.000	1.555	4	14	0
4598	0.30	0.00	0.30	0.0	0.00	0.00	0.00	0.00	0.00	0.00	...	0.102	0.718	0.0	0.000	0.000	0.000	1.404	6	118	0
4599	0.96	0.00	0.00	0.0	0.32	0.00	0.00	0.00	0.00	0.00	...	0.000	0.057	0.0	0.000	0.000	0.000	1.147	5	78	0
4600	0.00	0.00	0.65	0.0	0.00	0.00	0.00	0.00	0.00	0.00	...	0.000	0.000	0.0	0.125	0.000	0.000	1.250	5	40	0

4601 rows × 58 columns

Amount of Data

According to paper 1, the dataset has 34519 records, with a ratio of spam to non-spam as 1:3. The author mentioned that duplicates elimination and data balancing are applied to the dataset, so the actual data are less than 34519 records. Although the paper did not mention the amount of data after processing, it is still far more than the amount of data in paper 2, which has only 4601 records. The paper 2 dataset is around eight times smaller than the dataset of paper 1.

Machine Learning Model

Paper 1 used a deep learning model, while paper 2 used a decision-tree-based ensemble model. Deep learning models can work with many different data types, such as tabular data, text data, or even image data and audio data. Meanwhile, Tree-based models can only work with tabular data, which is data in a table format. Deep learning models have many essential hyperparameters to tune, such as the learning rate, the activation function, number of layers, the number of nodes in each layer, the type of layer, number of epochs, batch size, loss function, and optimizer etc. In comparison, a tree-based model has only a few essential hyperparameters, such as the number of estimators, the maximum depth of the tree, the number of features to consider when looking for the best split etc.

Model Training Time and Testing time

Paper 1 did not mention the training time of the LSTM and GRU models, so we made an educated guess by looking at the number of parameters each model has to train. The total number of parameters is 57,476,974 for the LSTM model, 57,436,874 for the GRU model. Such a vast number of parameters must require a long training time. We predict the training time will take many hours or even days, depending on the quality level of the GPU. Although Paper 1 did not mention the training time, it did mention the testing time on the entire testing dataset. According to Paper 1, the execution time of the LSTM and GRU in classifying the entire test data during the validation process is 45.267292 seconds and 44.55612 seconds, respectively.

Paper 2 did not mention both the training time and the testing time of the XGBoost model. So we have implemented a python program using the same model with the same hyperparameters under the same dataset environment. Using the ipython magic function '%%time', we get the average values of CPU times and Wall clock times for both training and testing. The average training CPU time is only around 5 seconds, while the average training Wall clock time only takes 1 second. The average testing CPU time only takes 26 milliseconds, while the average testing Wall clock time only takes 10 milliseconds.

From the training time and testing time of approaches from both papers, we can see both training time and testing time from paper 2 approaches take much faster than paper 1.

Model Performance Difference

According to paper 1, LSTM with dropout has achieved 99.02% accuracy and 98.92% recall. GRU with dropout has achieved 99.02% accuracy and 98.98 recall. In paper 2, XGBoost has achieved 96.88% accuracy and 95.59% accuracy. In this classification problem, it is essential to classify spam, so we most care about the recall. Comparing the recalls, the GRU and LSTM models perform much more accurately than XGBoost.

Since the two papers used a different dataset to train and test, it is not fair. Generally, the deep learning approach will produce higher accuracy than a decision tree-based ensemble model if given enough training data. Paper 1 also provided the performance of XGBoost for the same dataset. The accuracy of XGBoost is only 98.52%, and recall is only 98.17% in the Enron company dataset.

Summary

	Paper 1	Paper 2
Source of Dataset	Spam email dataset owned by the Enron company	Spambase dataset from UCI machine learning repository
Feature engineered	No	Yes
Datatype of features	Raw text files	Extracted numerical values
Amount of Data	More (34519)	Less (4601)
Model Type	Deep Learning	Decision-tree-based Ensemble
Model Used	LSTM / GRU	XGBoost
Cross Validation	7 fold cross validation	7:3 (training to testing ratio) with leave one out cross validation
Training time	Longer More than hours or even days, depending on the quality of GPU	Shorter Average Wall Time: 1.76 seconds Average CPU Time: 6.3 seconds

Testing time on entire test data	Slower Model with LSTM: 45.27 seconds Model with GRU: 44.56 seconds	Faster Average Wall Time: 10 milliseconds Average CPU Time: 26.4 milliseconds
Performance	Very High LSTM Accuracy: 99.02% LSTM Recall: 98.92% GRU Accuracy: 99.02% GRU Recall: 98.98%	High XGBoost Accuracy: 96.88% XGBoost Recall: 95.59%

The Advantages and Disadvantages of the two proposed solutions

Using the deep learning approach (LSTM/GRU) in the spam email detection problem, we need to acquire and raw data and mark them whether spam or ham. We can save time from doing feature engineering because we can feed our data to the pre-trained word embedding layer and LSTM or GRU layer. We will also get the best performance than other traditional machine learning methods if we were given enough data.

However, there are also disadvantages of deep learning models. It requires the expert to tune many hyperparameters. Each hyperparameter is critical, and a lousy hyperparameter will cause poor performance. It also requires much more data compared to a decision tree-based ensemble model. The computation time in both training and testing of the deep learning model is also much longer than a decision tree-based ensemble model. RNNs are extremely expensive for processing long sequences [6]. Thus the LSTM/GRU model proposed from paper 1 will take even more computational time than other deep learning models.

When using the decision tree-based ensemble approach (XGBoost) in the spam email detection problem, they are relatively easier to use because it requires fewer hyperparameters tuning and less computationally expensive. Its training and testing are also much faster than a deep learning model. Most importantly, it requires much less data than a deep learning model.

The only downside is that decision tree-based ensemble approach can only work with tabular data. This means we need to prepare a well-featured engineered dataset instead of just feeding the text data. XGBoost has produced a good accuracy in this spam email detection problem, but it is not as good as the LSTM/GRU deep learning model.

To conclude, both proposed solutions from the two papers have their advantages and disadvantages. When we want to apply a machine learning approach to solve a problem, we must carefully choose an appropriate machine learning method by considering the amount of data, computational resources, and time.

Our Proposed Method

The proposed solutions of the two papers already achieve remarkably high accuracy. However, we found that the deep learning approach of paper 1 involves too many parameters to train. We have come up with the idea that solves the problem more efficiently by utilizing 1D convolutions. 1D convolutions can help us to extract local 1D subsequences from long sequences [7]. Since LSTM/GRUs are extremely expensive for processing very long sequences, our proposed idea will use 1D convolution and 1D pooling to shorten the sequence and extract useful representations as a preprocessing step.

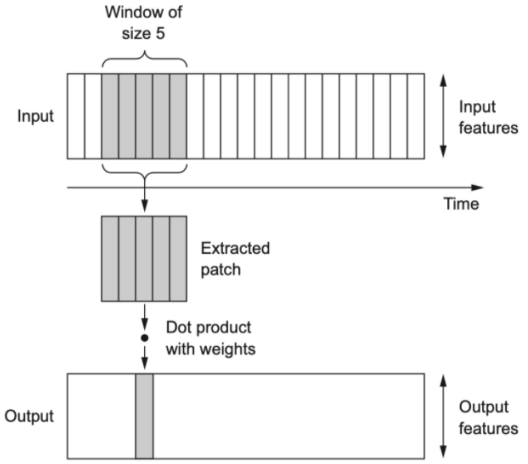


Figure A: How 1D convolution works

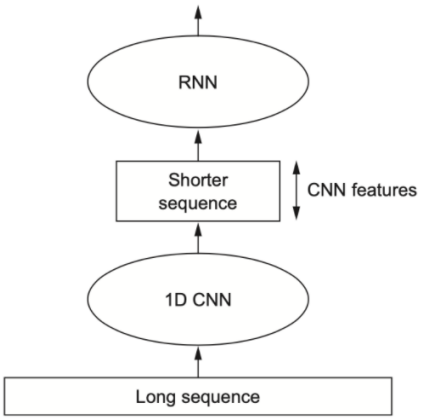
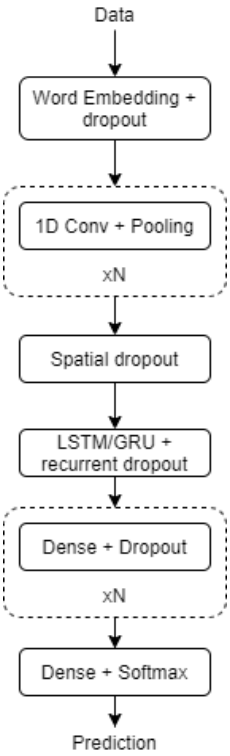


Figure B: Combining a 1D convnet and an RNN for processing long sequences

To further reduce the input sequence length, we can introduce a 1D average pooling or 1D max-pooling after the 1D convolution. Thus the model will be more efficient. Here is the model of our proposed idea.



Reference

[1] Basyar, Iqbal, Adiwijaya, and Murdiansyah, Danang Triantoro, “Email Spam Classification Using Gated Recurrent Unit and Long Short-Term Memory,” *Journal of computer science*, vol. 16, no. 4, pp. 559–567, 2020, doi: 10.3844/jcssp.2020.559.567.

[2]Sepp Hochreiter and J¨urgen Schmidhuber. Long Short-Term Memory. Neural computation, 9(8):1735–1780, 1997.

[3] Dheeru Dua and Casey Graff.UCI Machine Learning Repository. 2017. Url:http://archive.ics.uci.edu/ml.

[4] Mustapha, Ismail B, Hasan, Shafaatunnur, Olatunji, Sunday O, Shamsuddin, Siti Mariyam, and Kazeem, Afolabi, “Effective Email Spam Detection System using Extreme Gradient Boosting,” 2020.

[5] “Introduction to Boosted Trees,” Introduction to Boosted Trees - xgboost 1.5.0-SNAPSHOT documentation. [Online]. Available: https://xgboost.readthedocs.io/en/latest/tutorials/model.html. [Accessed: 26-Apr-2021].

[6] Chollet François, “Working with text data,” in Deep Learning with Python, Second Edition, Shelter Island, NY: Manning Publications, pp. 196–223.

[7] L. Kaiser, A. N. Gomez, and F. Chollet, ‘Depthwise Separable Convolutions for Neural Machine Translation’, *ArXiv*170603059 Cs, Jun. 2017, Accessed: Apr. 26, 2021. [Online]. Available: http://arxiv.org/abs/1706.03059.